PickyRTL: A Static Analysis Tool for Detecting Hardware CWEs at Register-Transfer Level

Advisor: Dr. Boyang Wang

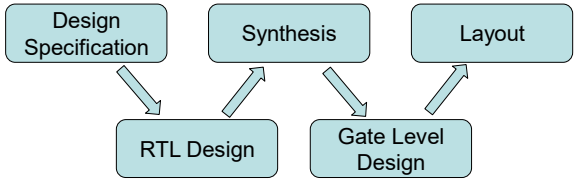
John Parks

University of Cincinnati

Background

Hardware vulnerabilities cannot be fully fixed with software patches or microcode updates alone. They often require physical modifications to the chip itself, driving up costs and leaving systems exposed for longer periods. Meltdown and Spectre demonstrated this by exploiting speculative execution in processors to steal sensitive data and affected nearly every device on the market. This is why catching vulnerabilities before fabrication is critical. The perfect time for that is during RTL design.

Register Transfer Level (RTL) design is a stage in the digital circuit design process that describes the behavior and structure of a system. It defines how data moves between registers and the operations performed on that data. RTL designs are written using **hardware description languages (HDLs)** such as Verilog, SystemVerilog, or VHDL. Because RTL is expressed as code, it can be analyzed using static analysis techniques.



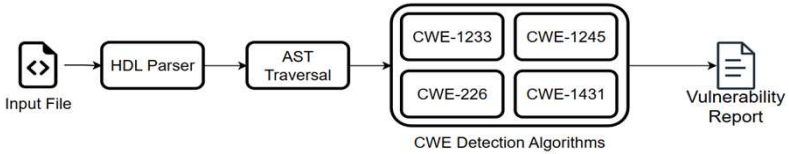
What is CWE?

Common Weakness Enumeration (CWE)
Community-developed list of common software and hardware weaknesses that could lead to vulnerabilities

Four CWEs selected for detection:

- **CWE-1245:** Improper Finite State Machines (FSMs) in Hardware Logic
- **CWE-1233:** Security-Sensitive Hardware Controls with Missing Lock Bit Protection
- **CWE-226:** Sensitive Information in Resource Not Removed Before Reuse
- **CWE-1431:** Driving Intermediate Cryptographic State/Results to Hardware Module Outputs

Detection Algorithms



CWE-1245 Detection

1. Identify the state variable for FSM
2. Convert assignments to the state variable into FSM transitions
3. Search for
   - Incomplete State Coverage
   - Unreachable States
   - Deadlock States

```
module fsm (input clk, input x);
    reg [1:0] state;
    always @(posedge clk) begin
        case (state):
            0: state = 2;
            1: state = 1;
            2: state = x ? 1 : 3;
            3: state = 0
        endcase
    end
endmodule
```

```
module unprotected_key (input clk,
input reglk, input data);
    reg key0, key1;
    always @(posedge clk) begin
        key0 = data;
        key1 = reglk ? key1 : data;
    end
endmodule
```

CWE-1233 Detection

1. Identify lock bit registers
2. Identify security-sensitive registers
3. Gather assignments for each security-sensitive register
4. Ensure each assignment is protected with the lock bit

CWE-226 Detection

1. Identify reset signals
2. Identify sensitive registers that need to be reset
3. Ensure there is an assignment that resets the register within a conditional containing the reset register

```
module hash_msg (input clk, input rst,
input msg, output hash);
    always @(posedge clk) begin
        if (rst) begin
            hash = 0
            // msg never cleared
        end
        //...hashing logic
    end
endmodule
```

```
module crypto (input clk, input
data_i, output data_o);
    reg result, result_valid;
    always @(posedge clk) begin
        //...cryptographic operation
    end
    assign data_o = result
endmodule
```

CWE-1431 Detection

1. Identify cryptographic modules
2. Identify the output wire containing the result
3. Ensure assignments driven to the output wire are only made when the result is valid

Results

Evaluated on a dataset of 39 Verilog and System Verilog files

CWE	# Vulnerable Components	TP	FP	FN	Precision	Recall
CWE-1245	9	9	0	0	100.0%	100.0%
CWE-1233	30	10	9	11	67.9%	63.3%
CWE-226	31	21	1	10	95.5%	67.7%
CWE-1431	10	10	0	0	100.0%	100.0%
Total	**80**	**59**	**10**	**21**	**85.5%**	**73.8%**

Strong performance for CWE-1245 & CWE-1431 where weaknesses appear in structural patterns

Weaker performance for CWE-1233 & CWE-226 where more semantic context is required

Limitations

Limited Dataset Size
While we have preliminary results from our dataset, a larger, more diverse dataset would enable a more comprehensive evaluation.

Identifying Security-sensitive Registers
With static analysis, we only have a limited amount of context about the module and the defined registers. Without more semantic context, it is hard to identify security-sensitive registers

Name Matching
Identifying cryptographic modules, reset signals, and lock bits with name matching works for conventional names. However, this technique breaks down when non-conventional names are used

Future Work

We are actively working to improve PickyRTL:

- Creating a larger dataset
- Improvements to each CWE detection algorithm
- Extending coverage to additional CWEs